Universidad Nacional Autonoma de México

Proyecto de Microncontroladores

# NVIC y GPIO

*Cruz Morales Pablo Osvaldo*

5 de marzo de 2026

# Índice

**1. Planteamiento del proyecto** **3**

1.1. Descripción del proyecto . . . . . 3

1.2. Metodología . . . . . 3

1.3. Requerimientos . . . . . 3

**2. Marco Teórico** **3**

**3. Diseño a nivel sistema** **5**

3.1. Diseño a nivel sistema del GPIO . . . . . 6

3.2. Diseño a nivel sistema del NVIC . . . . . 7

**4. Diseño a detalle** **9**

4.1. Habilitar y configurar el gpio . . . . . 9

4.2. Habilitar la interupción . . . . . 11

4.3. Monitoreo del flanco de subida del GPIO . . . . . 12

4.4. Pausar el GCM . . . . . 13

4.5. Almacenar cuenta del contador del programa y el valor actual del ACC. . . . . 15

4.6. Cargar la dirección de la rutina de interrupción en el PC. . . . . 18

4.7. Seleccionar la dirección guardada,complemtnar el valor de load del PC y reanudar el funcionamiento del GCM . . . . . 18

4.8. Ejecutar la rutina de interrupción y después limpiar la bandera de interrupción. . . . . 20

4.9. Pausar el GCM . . . . . 21

4.10. Modificar el selector del bloque M,para que a la salida se refleje el ACCnvic . . . . . 21

4.11. Cargar la dirección de la rutina de interrupción en el PC y actulizar el valor de ACC. 22

4.12. Modificar el selector del bloque M,para que a la salida se refleje el ACCtemp y reseteo de un registro relacionado al GPIO . . . . . 23

4.13. Seleccionar la dirección de la rutina de interrupcción,mover el valor de 1 a 0 del pin load del PC y reanudar el funcionamiento del GCM . . . . . 24

4.14. Monitorear flanco de subida del GPIO . . . . . 24

**5. Maquina de estado del NVIC** **25**

5.1. Maquina de estado del NVIC en Quartus . . . . . 27

**6. Instrucciones** **27**

**7. Resultados** **28**

7.1. GPIO modo entrada . . . . . 28

7.2. GPIO modo salida . . . . . 28

7.3. GPIO como interrupción . . . . . 28

**8. Conclusiones** **29**

# 1. Planteamiento del proyecto

Se busca establecer una estructura que, al habilitarse la interrupción, configure adecuadamente el GPIO para pausar el GCM, almacenar la dirección de la instrucción en la que surgió la interrupción, así como el valor del ACC. Luego, se cargará la dirección de la rutina de interrupción en el PC, se reanudará el funcionamiento del GCM, se ejecutará la rutina de interrupción, se limpiará la bandera de interrupción y se restaurarán la dirección y el valor del ACC al PC y ACC, respectivamente. Finalmente, se procederá a despausar y continuar con la siguiente instrucción después de la interrupción.

## 1.1. Descripción del proyecto

En este proyecto, se llevará a cabo la implementación del comportamiento descrito en el planteamiento del proyecto, técnicamente conocido como NVIC , partiendo de un procesador previamente diseñado. El objetivo es integrar y configurar el NVIC para gestionar eficientemente las interrupciones en el sistema, mejorando así la capacidad de respuesta y la estabilidad del procesador en entornos de tiempo real.

## 1.2. Metodología

La metodología basé para este proyecto será la Metodología de diseño - Ulrich.

## 1.3. Requerimientos

Después de analizar detenidamente el planteamiento del proyecto, se plantea en forma de requisitos específicos que aseguran la integración efectiva y el rendimiento óptimo del NVIC:

- **incorporación de nuevos registros referente a la configuracion**
- **Modificación del GCM**
- **Modificación del PC**
- **Implementación de un GCM de interrupciones**
- **Implementación del GPIO**

# 2. Marco Teórico

Para el desarrollo de este proyecto, fue de suma importancia comprender a fondo los siguientes conceptos y principios fundamentales en la construcción de un microprocesador:

- **Mapas de Karnaugh:** Los mapas de Karnaugh son una herramienta utilizada en diseño digital para simplificar funciones lógicas complejas. Se representan en forma de tablas que muestran todas las combinaciones posibles de entradas y sus correspondientes salidas lógicas. Estos mapas se utilizan para minimizar circuitos lógicos y reducir la complejidad de las funciones lógicas.

- **Flip-Flops:** Los flip-flops son dispositivos secuenciales que se utilizan para almacenar un bit de información. Existen diferentes tipos de flip-flops, como el D-flip-flop y el JK-flip-flop, que se utilizan en la construcción de registros y memorias. Un flip-flop tiene dos estados estables (0 y 1) y se utiliza para almacenar un valor binario.

- **Registros:** Un registro es un conjunto de flip-flops utilizados para almacenar datos temporales en un microprocesador. Los registros son esenciales para realizar operaciones y cálculos. En un microprocesador, encontrarás registros generales, registros de dirección y otros tipos de registros especializados.

- **Memorias:** Las memorias son componentes que almacenan datos de manera persistente. En un microprocesador, se utilizan dos tipos de memorias: la memoria de programa (ROM o Flash) que almacena el código de las instrucciones y la memoria de datos (RAM) que almacena datos temporales.

- **Contador:** Un contador es un componente que lleva un registro de la dirección de memoria actual en la que se encuentra una instrucción. El contador de programa (PC) es fundamental para determinar qué instrucción se ejecutará a continuación. Controla el flujo de ejecución del programa.

- **Decodificador:** El decodificador es un componente que interpreta las instrucciones almacenadas en la memoria de programa y las traduce en señales de control que indican a la CPU qué operación debe realizar. Es esencial para la ejecución correcta de las instrucciones.

- **Unidad Lógica y Aritmética (ALU):** La ALU es el corazón del microprocesador y realiza operaciones aritméticas y operaciones lógicas en los datos almacenados en los registros. Es responsable de realizar cálculos y operaciones lógicas en el procesador.

- **Importancia del Contador de Instrucciones (PC):** El contador de instrucciones (PC) es crítico para la ejecución de un programa, ya que indica la dirección de memoria de la próxima instrucción que debe ejecutarse. Controla la secuencia de instrucciones y asegura que el programa se ejecute en el orden correcto.

- **Unidad de Control:** La unidad de control es responsable de secuenciar y coordinar todas las operaciones del microprocesador. Genera señales de control que dirigen la ejecución de las instrucciones y la operación de los diferentes componentes del procesador.

- **Bus de Datos y Bus de Dirección:** Los buses de datos y de dirección son canales de comunicación utilizados para transferir información entre diferentes componentes del microprocesador, como registros, ALU y memoria. El bus de datos transporta los datos, mientras que el bus de dirección indica la ubicación de memoria o registro.

- **Conjunto de Instrucciones :** El conjunto de instrucciones define el conjunto de operaciones que el microprocesador puede ejecutar. Comprender el ISA es esencial para el diseño de un procesador y su programación.

# 3. Diseño a nivel sistema

Teniendo en cuenta el planteamiento del problema, podemos simplificarlo en tres bloques principales: GPIO, NVIC y PROCESADOR. Los primeros dos bloques son configurables por el procesador según su uso. A partir de este análisis, se obtiene el siguiente diseño a bloques:

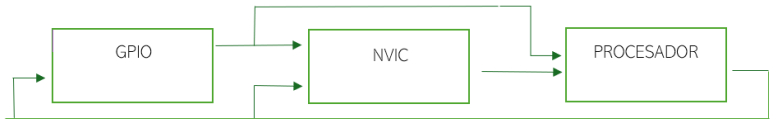


Figura 1: Diseño a bloques

El procesador configura el GPIO y el NVIC. Para configurar el GPIO, primero se debe habilitar el reloj (CLK) del GPIO. Luego, se elige la dirección del GPIO, que puede ser de entrada o salida.

- **En el caso de configuración como entrada**: La línea de datos pasa del GPIO al procesador, donde el valor se almacena en un registro.
- **En el caso de configuración como salida**: Los datos pasan del procesador a los registros del GPIO, permitiendo visualizar los datos en los pads externos.

El NVIC también es configurado por el procesador, el cual controla el estado del vector de habilitación de interrupciones. En este proyecto, solo se trabaja con una interrupción, por lo que el vector se reduce a un registro.

El bus de datos que entra al NVIC está relacionado con el GPIO, porque el NVIC está monitoreando funcionamiento de un flanco de subida obtenido de una variación en el GPIO. Por lo tanto, es indispensable su previa configuración en modo de entrada.

Cuando el NVIC detecta un flanco de subida, se retiene un valor 1, iniciando el proceso de interrupción. El proceso de interrupción incluye los siguientes pasos:

- Pausar el GCM
- Almacenar cuenta del contador del programa y el valor actual del ACC.
- Cargar la dirección de la rutina de interrupción en el PC.
- Reanudar el funcionamiento del GCM.
- Ejecutar la rutina de interrupción.
- Limpiar la bandera de interrupción.
- Restaurar la cuenta con un incremento de una unidad y el valor del registro temporal al PC y ACC, respectivamente.
- Despausar el sistema y continuar con la ejecución de la siguiente instrucción después de la interrupción.

## 3.1. Diseño a nivel sistema del GPIO

El GPIO cuenta con las siguientes entradas y salidas

Figura 2: Entidad

El enable CLK sirve para habilitar el uso del GPIO y por tanto poder definir si será usado con datos de entrada o salida.

- **En el caso de configuración como entrada**: La línea de datos pasa del GPIO al procesador, donde el valor se almacena en un registro.
- **En el caso de configuración como salida**: Los datos pasan del procesador a los registros del GPIO, permitiendo visualizar los datos en los pads externos.

El CLK está relacionado con modo entrada porque los datos que llegan al registros se guardan en el momento que llega un flanco de subida por el señal CLK.

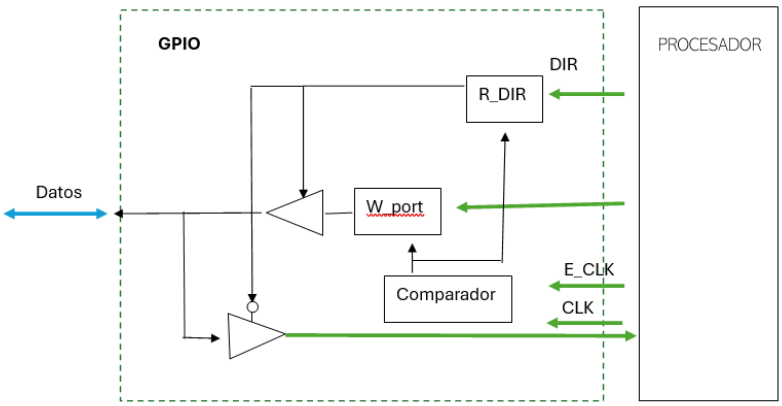


Figura 3: Diseño a bloques del GPIO

Como se mencionó anteriormente, los datos siguen dos flujos configurables. Si se define el GPIO como entrada, los datos se transmiten directamente al procesador. Por otro lado, si ocurre lo contrario,

es decir, si se configura el GPIO como salida, los datos fluyen desde el procesador hacia los registros, para finalmente ser mostrados en los pines de salida.

## 3.2. Diseño a nivel sistema del NVIC

Las entradas y salidas del NVIC son :

Figura 4: Entidad

El ACC se transfiere al NVIC para respaldar su valor una vez que el GCM ha sido pausado. Este respaldo permanece hasta que la bandera de interrupción se limpie, momento en el cual el valor guardado se carga de vuelta al ACC.

El proceso de interrupción se inicia mediante el GPIO, siempre y cuando esté habilitado y configurado en dirección de entrada con la interrupción habilitada.

Cuando el NVIC detecta un flanco de subida, se inicia un proceso que se resume en:

- Pausar el GCM
- Almacenar cuenta del contador del programa y el valor actual del ACC.
- Cargar la dirección de la rutina de interrupción en el PC.
- Reanudar el funcionamiento del GCM.
- Ejecutar la rutina de interrupción.
- Limpiar la bandera de interrupción.
- Resta urar la cuenta con un incremento de una unidad y el valor del registro temporal al PC y ACC, respectivamente.
- Despausar el sistema y continuar con la ejecución de la siguiente instrucción después de la interrupción.

De forma interna las señals se relacionan de la soguiente forma:

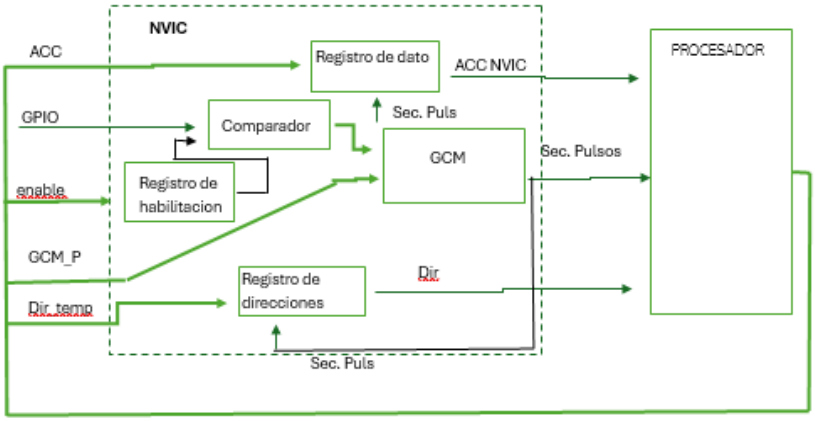


Figura 5: Diseño a bloques del NVIC

Las entradas, como el GPIO y el Enable, están asociadas a la configuración del GPIO como entrada y la habilitación de la interrupción. Por otra parte, se comparte un bus de datos con el ACC, ya que justo después de iniciar la interrupción y pausar el GCM, el NVIC guarda el dato del ACC y la cuenta del PC (dirección). Para las salidas, la secuencia de pulsos está compuesta por dos señales: la primera se utiliza para pausar el GCM y la segunda para enviar una señal al contador del programa.

La primera vez que se envía la segunda señal, se carga la dirección de la rutina de interrupción. En la siguiente ocasión, esta señal carga la dirección de la instrucción siguiente a la que estaba ejecutándose antes de la interrupción.

Por último, la señal ACCnvic proporciona el valor que fue almacenado en el ACC cuando se inició la interrupción, esta señal se usa despues de haber ejecutado la rutina de interrupción.

Ahora al profundizar el bloque del procesador se obtiene:

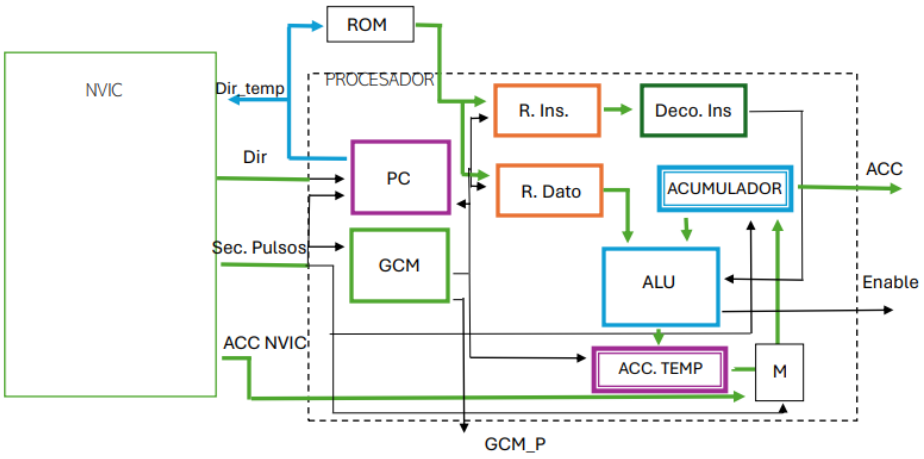


Figura 6: Diseño a bloques del Procesador

Como se observa, la señal Dir entra al PC,que es donde se lleva la cuenta del programa,tambíen se tiene una entrada que es parte de la secuencia de pulsos, que llegará después de garantizar la pausa del GCM. La señal Accnvic son la entrada del bloque M, que de forma tecinca, el bloque M, es un multiplexor 2 a 1, que se encarga que antes de finalizar el proceso de interrupcion el ACC tenga a la entrada el ACC previamente guardado, esta situación con un pulso de carga en un estado siguiente garantiza la introduccipon del dato al ACC.

# 4. Diseño a detalle

El diseño a detalle se irá desglozando por cada tarea en la rutina de interupción descrita a continuación.

- Habilitar y configurar el GPIO
- Habilitar la interupción
- Monitorear flanco de subida del GPIO
- Pausar el GCM
- Almacenar cuenta del contador del programa y el valor actual del ACC.
- Cargar la dirección de la rutina de interrupción en el PC.
- Seleccionar la dirección guardada,complemtnar el valor de load del PC y reanudar el funcionamiento del GCM.
- Ejecutar la rutina de interrupción y después limpiar la bandera de interrupción.
- Pausar el GCM
- Modificar el selector del bloque M,para que a la salida se refleje el ACCnvic
- Cargar la dirección de la rutina de interrupción en el PC y actulizar el valor de ACC.
- Modificar el selector del bloque M,para que a la salida se refleje el ACCtemp y reseteo de un registro relacionado al GPIO
- Seleccionar la dirección de la rutina de interrupción,mover el valor se 1 a 0 del pin load del PC y reanudar el funcionamiento del GCM.
- Monitorear flanco de subida del GPIO

## 4.1. Habilitar y configurar el gpio

Las entradas y salidas son las presentadas en la **Figura 7.** ahora estan relacionadas a dos nuevos bloques como se ve a continuación.

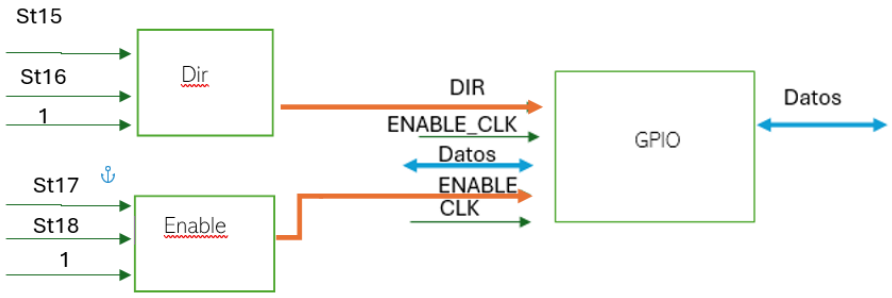


Figura 7: Bloques de configuraicón del GPIO

El estado del GPIO es constante por el uso de registros que guardan la configuración de Dir y Enable.

La estructura lógica de ambos bloques consiste en registros tipo D. Las instrucciones St15 y St17 están conectadas al pin CLK, mientras que las instrucciones St16 y St17 están conectadas al pin de reset. La entrada del registro D es un valor alto. Por ejemplo, al usar la instrucción St15, el registro guarda un valor de 1, habilitando el GPIO. Por el contrario, al ejecutar la instrucción St18, el registro se resetea, obteniendo un 0 en la salida, lo que deshabilita el GPIO.

Ahora, la misma estrutura pero vista en el formato de Quartus.

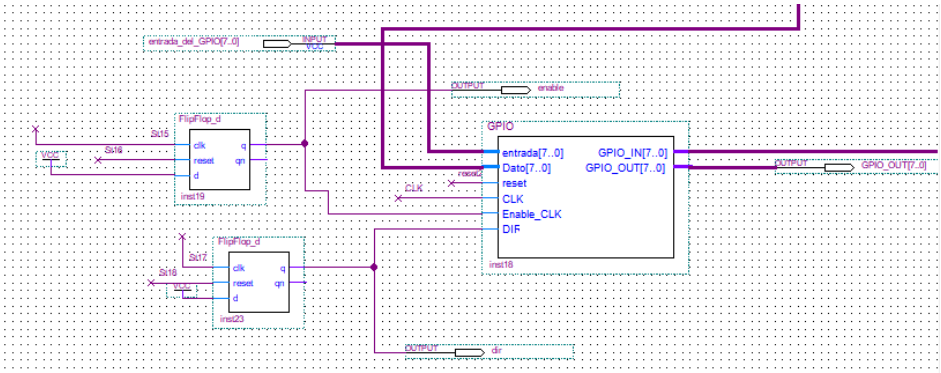


Figura 8: Bloques de configuraicón del GPIO

De forma interna el GPIO parte de una estructura basica:

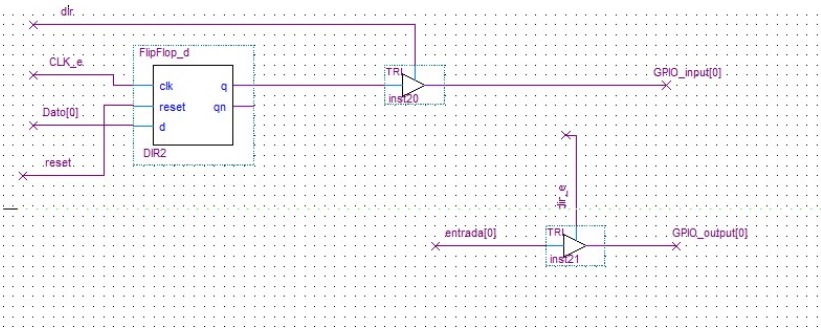


Figura 9: SDiseño GPIO por bit

Este diseño se repite 8 veces: hay un registro por cada bit, y juntos almacenan los 8 bits que salen del procesador y se reflejan en los pads del GPIO cuando este está configurado como salida.

## 4.2. Habilitar la interupción

Las entradas y salidas para esta tarea se presentaron anteriormente en la **Figura 5**, por lo que ahora se utilizará únicamente la parte de interes.

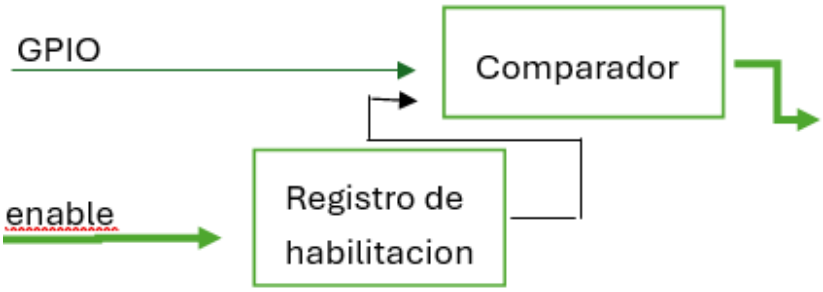


Figura 10: Bloque de Habilitación de interupción

Este bloque es una parte interna del NVIC que usa dos de las cinco señales que entran al NVIC. El propósito del NVIC es iniciar el proceso de interrupción cuando reciba un flanco de entrada proveniente del GPIO. Sin embargo, este proceso debe ser habilitado solo cuando sea necesario utilizarlo. Para habilitar la interrupción, se debe guardar un nivel alto en el registro de habilitación. Esto permite que el valor del GPIO de entrada se refleje en la salida del comparador. De este modo, si hay un flanco de subida, se iniciará el proceso de interrupción.

Ahora estos bloques serán descritos en Quartus.

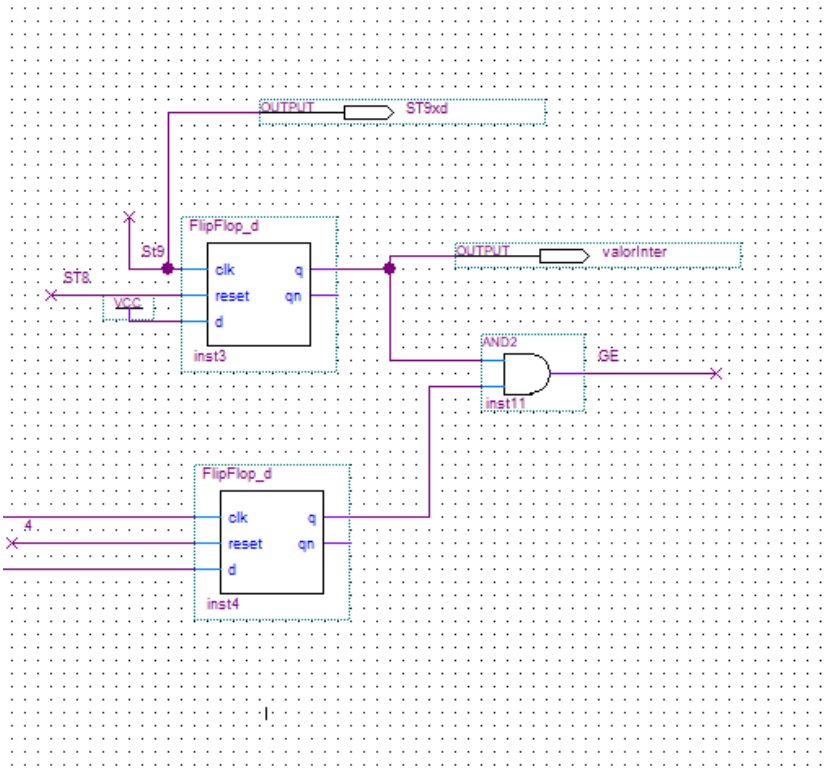


Figura 11: CLKG y Enable como entrada a una compurta AND

Para guardar un valor alto se necesita ejecutar la instrucción St9 y para poner un valor bajo la rutina St8. El ccomportamiento descrito previamente del comparador, es el de una compuerta AND.

## 4.3. Monitoreo del flanco de subida del GPIO

Una de las entradas del NVIC es el GPIO. El GPIO pasa por una compuerta OR, lo que permite que cualquier variación en la entrada sea detectada por una pequeña máquina de estados. Esta configuración permite retener el flanco de subida. Una vez retenido el flanco de subida, el valor se compara con la parte de habilitación previamente explicada.

Esto se describe en VHDL de la siguiente manera:

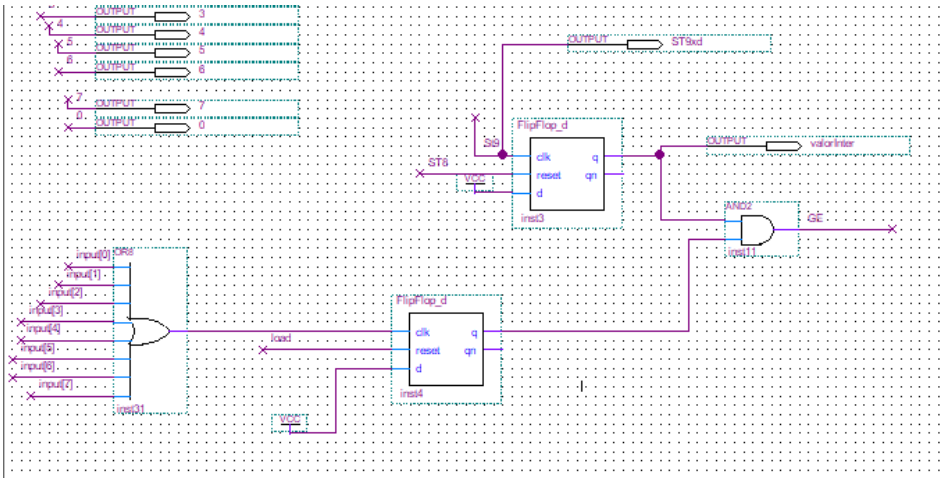


Figura 12: Esquematico Habilitación de interupcción

Se observa la compuerta OR conetada a los pines del GPIO. La maquina de estado deja de tener un valor 1 hasta que es reseteada.

## 4.4. Pausar el GCM

Dentro del proceso de interrupción, al guardar registros o cargar valores en el PC, así como al realizar otras acciones, es necesario asegurar que el GCM del procesador esté pausado para evitar errores en el proceso de interrupción. Para pausar el GCM se plantea la siguiente estructura.

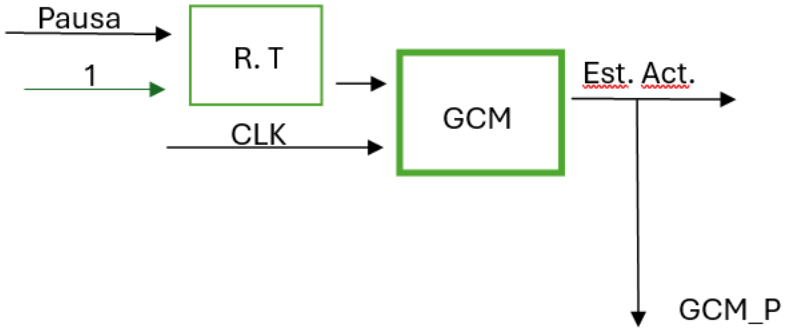


Figura 13: Bloque de Habilitación de interupción

Pausa es una señal que proviene del NVIC y se dirige al CLK del registro Toggle y la señal de alto va a la entrada del registro. El comportamiento de un registro T es que, si en la entrada hay un valor alto, cada vez que haya un flanco de subida en el CLK, el valor de salida será el complemento del dato anterior. Por lo tanto, en el primer flanco de subida de la señal Pausa, el valor de salida se complementará obteniendo un valor alto, y en el segundo flanco de subida de la señal Pausa, el valor de salida cambiará a un estado bajo.

Como es necesario saber el momento en el que el GCm está en pausa la salida GCMp es una entrada del NVIC, El diagrama de estado del GCM es el sigueinte

se ilustrada en la**Figura 14**.

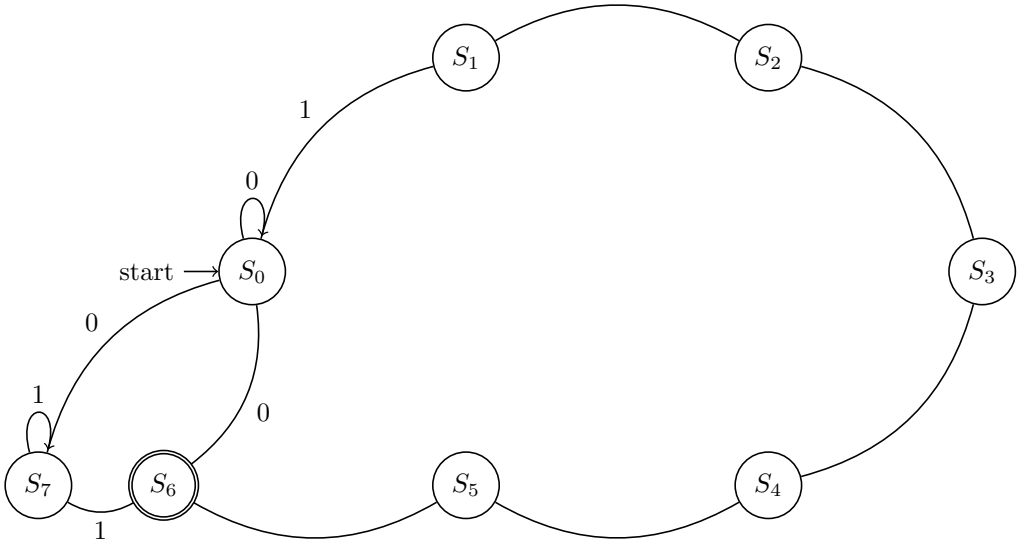


Figura 14: Diagrama de estados del GCM.

Como se puede observar, la señal de pausa se analiza cuando se está en el estado 6. Además, se mantendrá en el estado 7 siempre que la señal de pausa tenga un valor alto.

El GCM en Quartus es el siguiente.

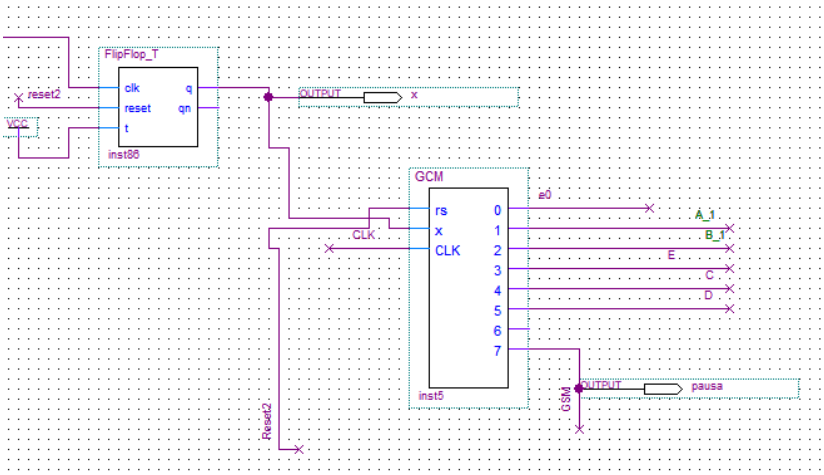


Figura 15: Bloque de GCM

## 4.5. Almacenar cuenta del contador del programa y el valor actual del ACC.

Después de recibir la señal del GCM confirmando el estado de Pausa, el NVIC pasa a un estado dos, en el cual se guardar la dirección del programa sumando 1, asi como el valor de registro del ACC.

La señal que reciben ambos registros son provenientes del NVIC asociados al estado 2 que entra a las señales CLK de ambos registros.

Para la primera se ilustra las entras y salidas mencionadas de la siguiente forma:

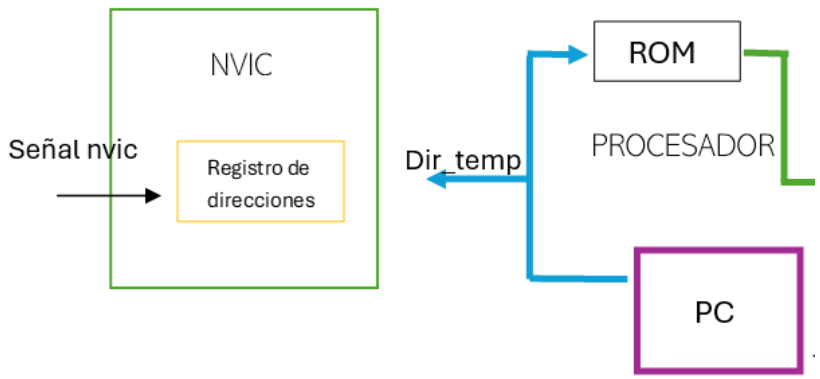


Figura 16: Bloque de GCM

El registro de direcciónes guardará el valor cuando el nvic de una señal de alto asociado al estado 2 y lo incrementará en 1. Este comprtamiento se describe en VHDL.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity registro_dir is
    Port (
        clk   : in  STD_LOGIC;
        reset : in  STD_LOGIC;
        d     : in  integer range 0 to 255;
        q     : out integer range 0 to 255
    );
end registro_dir;

architecture Behavioral of registro_dir is
begin

    process (clk, reset)
    begin
        if reset = '1' then
            q <= 0;
        elsif rising_edge(clk) then
            q <= d+1;
        end if;
    end process;
end Behavioral;
```

Figura 17: VHDL

La entrada es Q es la entrada de ladirección descrita en una variable entera de 0 a 255.

Al convertir el VHDL en un bloque e integrarlo se muestra la sigueinte Figura.

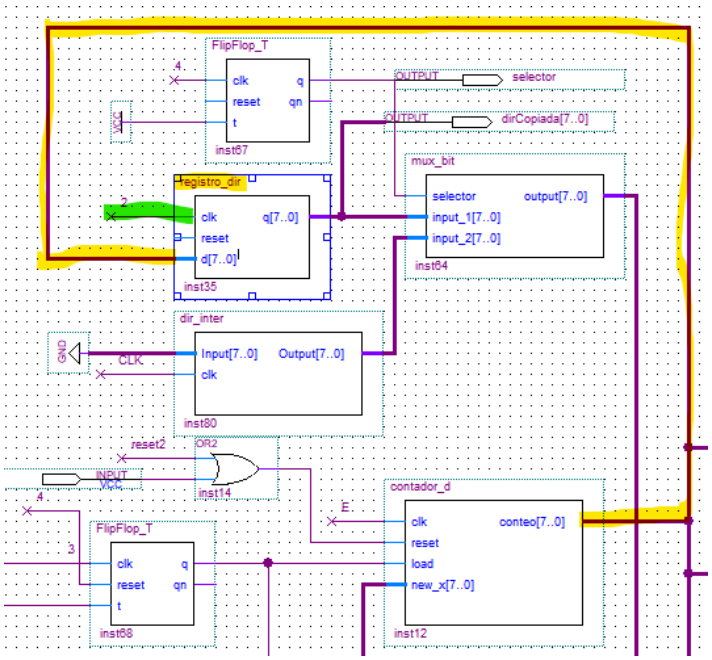


Figura 18: Esquematico para guardar direccion

Se puede observar como la salida del PC es la entrada del registro de direcciones, también que la señal 2 es la proveniente del NVIC. Para la segunda parte se plantea el la siguiente estructura.

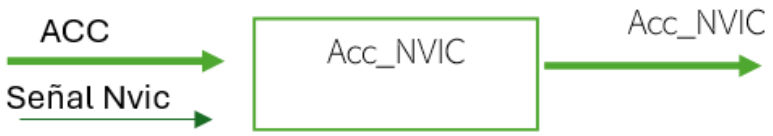


Figura 19: Diagrama Bloque

Este bloque está dentro del NVIC y recibe dos señales: una que viene del mismo NVIC y otra del ACC. LA señal que se recibe del NVIC sucede cuando se está en el estado 2.

En Quartus se ve de la siguente forma:

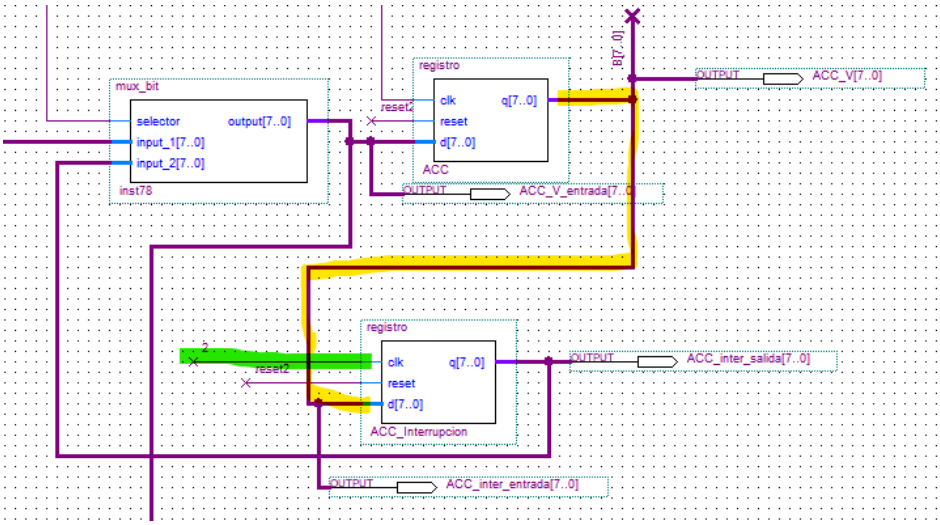


Figura 20: Bloque en Quartus

Se observa que la entrada proviene del ACC y que el CLK tiene asociada una señal 2 proveniente del NVIC.

## 4.6. Cargar la dirección de la rutina de interrupción en el PC.

Cuando el GCM está en pausa,pero ya se actualizaron los registros de ACCnvic y dir se pasa al estado 3. En este estado el PC recibe una señal del NVIC para cargar una direcciónn asociada a la rutina de interrupción. Este parte se ilustra como:

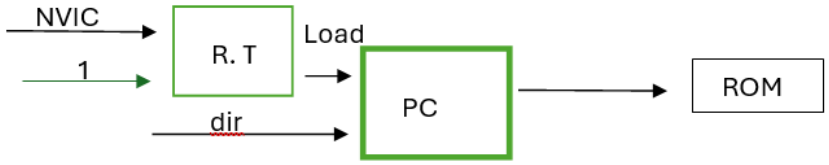


Figura 21: Bloque PC

Se vuuelve a usar un registro modo T, ahora la señal de NVIc está asociada al estado 3 y a la entrada una dirección proveniente del registro de direcciones.

En Quartus este bloque se ve de la siguiente forma:

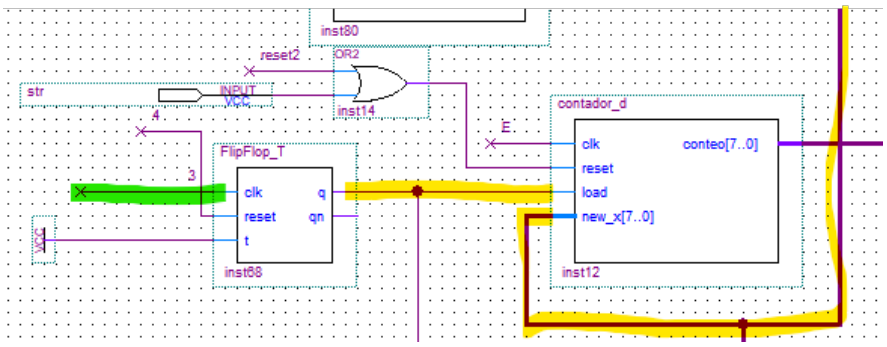


Figura 22: Bloque PC en Quartus

Se observa la entrada proveniente del registro de direcciones. Además, la señal de carga (load) está conectada a la salida del registro T y la señal 3 está conectada al CLK de entrada del registro T.

## 4.7. Seleccionar la dirección guardada,complemtnar el valor de load del PC y reanudar el funcionamiento del GCM

Despues de cargar direccion de la rutina de interupción se pasa a un estado 4 en el que el registro de direcciones reflejará en su salida la dirección más uno guardada en el estado 2, pasara de un estado alto a bajo el registro T asociado a la entrada load del PC y reanudará el funcinamiento haciendo del GCM haciendo lo mismo con su registro T asociado.

Para la primera parte

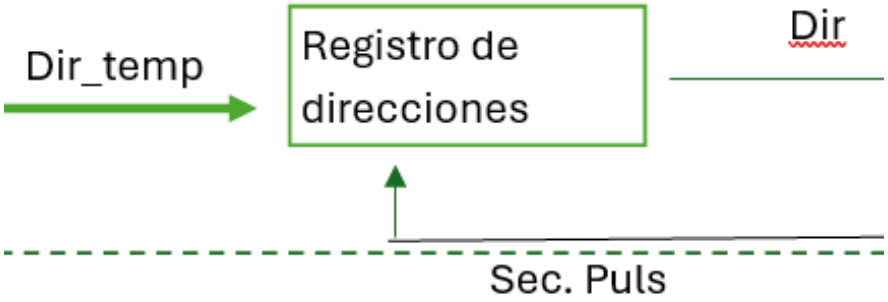


Figura 23: Bloque del registro de direcciones

La secuencia de pulsos moficiar el selector del registro de direcciones controlando la dirección de salida. Como se ve en Quartus:

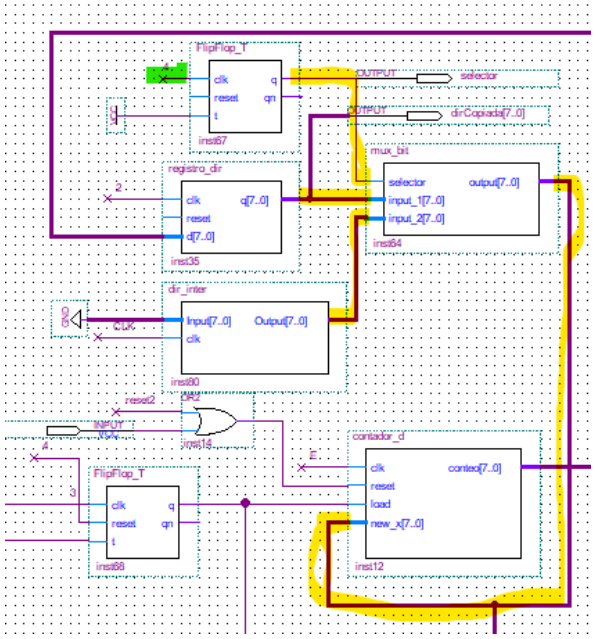


Figura 24: Bloque del registro de direcciones

La señal del NVIC controla la salida del registro T, para este estado pasa de 0 a 1, mostrando a la salida la direccion más uno guardada en el estado 2.

Para el valor de LOAD, se pasa de 1 a 0 reseteando el valor el registro T asociado con la señal 4.

Por ultimo, para despausar el GCM se complementa el valor de el registro T con un flanco de subida proveniente del estado 4.

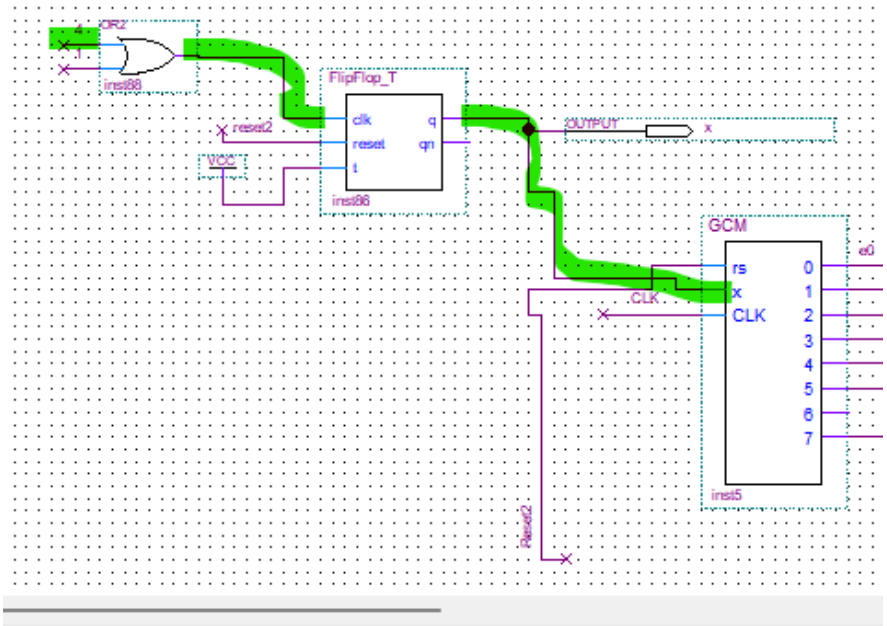


Figura 25: Bloque GCM en Quartus

El flanco de subida que llega al Registro T pasa por una compuerta OR, para que ambas entradas compartar una misma salida.

## 4.8. Ejecutar la rutina de interrupción y después limpiar la bandera de interrupción.

En este estado, el GCM está en despausa, ejecutando en la ROM la rutina de interrupción, al finalizar dicha rutina se ejecuta la instrucción para limpiar la bandera de interrupción. Como se ve a continuación:

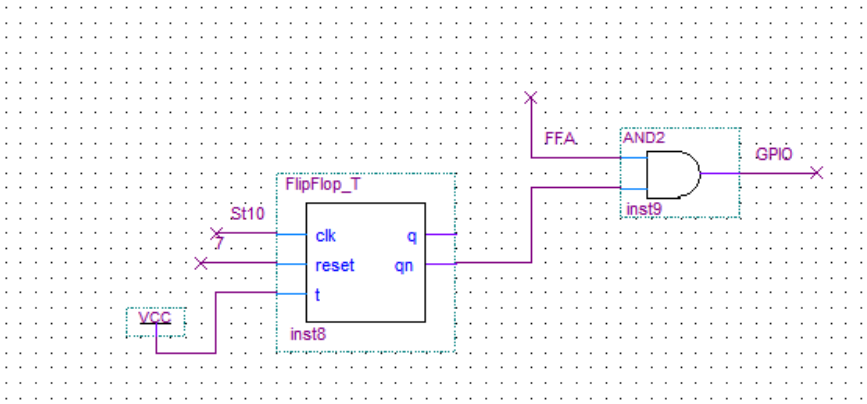


Figura 26: Bloque para limpieza de bandera

La instrucción que limpia la bandera es la St10. Esta instrucción complementa el valor del registro T, cambiándolo de 1 a 0, lo que a su vez permite que GPIO pase a 0

## 4.9. Pausar el GCM

Después de que GPIO valga 0, se pasa al estado 6 que es donde pauso el GCM, sin embargo este estado es el mismo que el 1, por lo que simplemente se repite.

## 4.10. Modificar el selector del bloque M,para que a la salida se refleje el ACCnvic

Ahora que se despauso el GCM y el GPIO está en 0, se pasa al estado 6.

En este estado se modifcará el bloque M del sigueinte esquema:

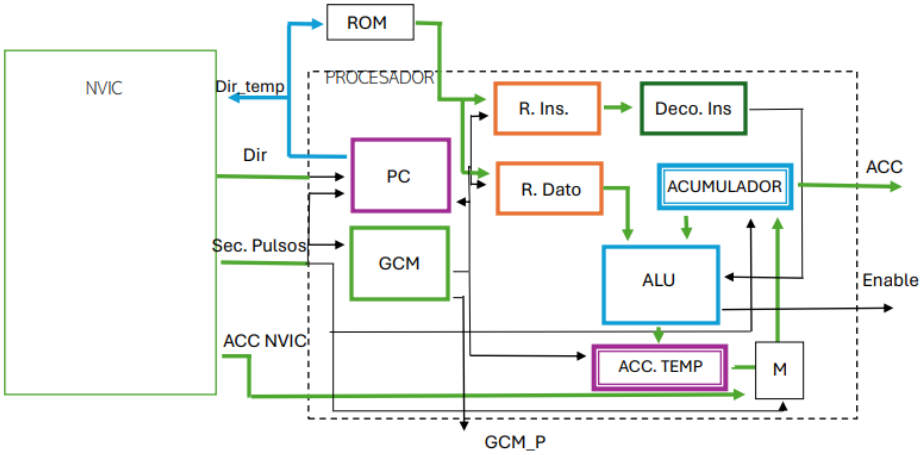


Figura 27: Diseño a bloques del Procesador

Al bloque M está definido como:

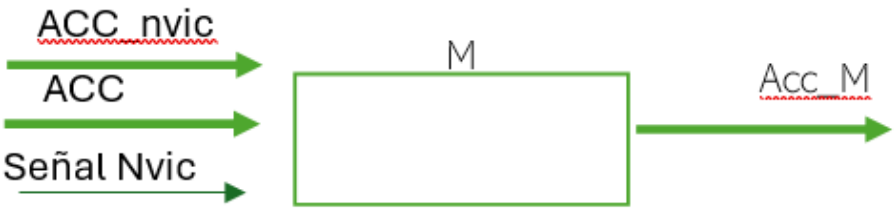


Figura 28: Diseño de bloque M

El bloque es un MUX, con dos entradas y una salida, la señal está asociada a un registro T, el cual conmuta por cada flanco de subida al CLK.

En quartus se tiene:

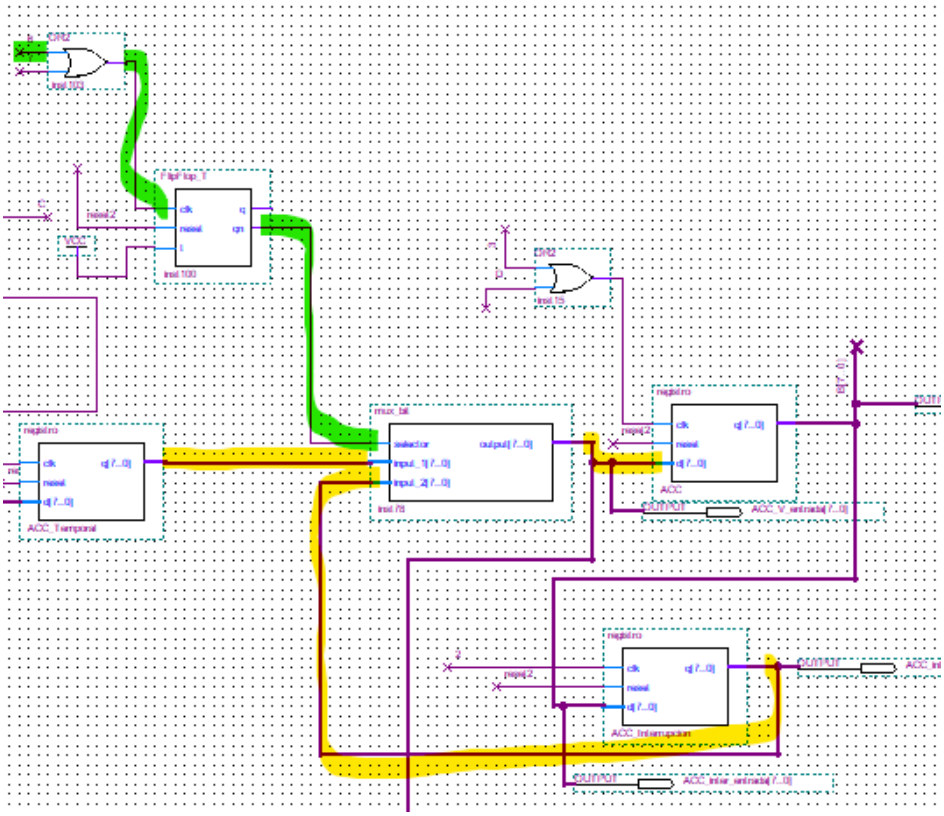

Figura 29: Diseño de bloque M en quartus

La entrada del registro $ACC$,está asocido a la salida del bloque M, esa salida es controlada por un registro tipo T. En el estado 6 conmuta por primera vez pasando de 1 a 0, conectando la salida del $ACC_{interrupciones}$ con la entrada del $ACC$.

## 4.11. Cargar la dirección de la rutina de interrupción en el PC y actulizar el valor de ACC.

Esta acción fue la misma hecha en el estado 3, por lo que se repite. Para la actualización del ACC, se tiene el sigueinte diagram en VHDL.

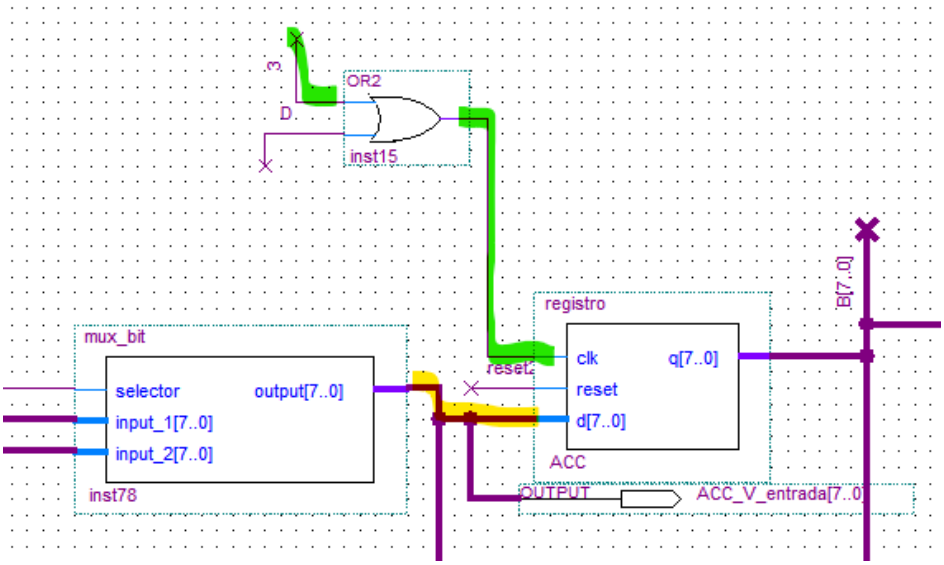


Figura 30: Diseño de bloque M en quartus

La primera vez que se ejecutó el Estado 3, el GCM estaba en pausa, lo que impidió que el ACCtem cambiara. Esto permitió que, al actualizar el ACC, su valor no variara. Sin embargo, para esta ocasión, al tener el ACCnvic en la entrada, se logra recuperar el valor respaldado en el estado 2.

## 4.12. Modificar el selector del bloque M,para que a la salida se refleje el ACCtemp y reseteo de un registro relacionado al GPIO

Después de cargar el Valor respaldado al ACC, se pasa a un estado 7, aqui el Selesctor de bloque M pasa de 0 a 1.

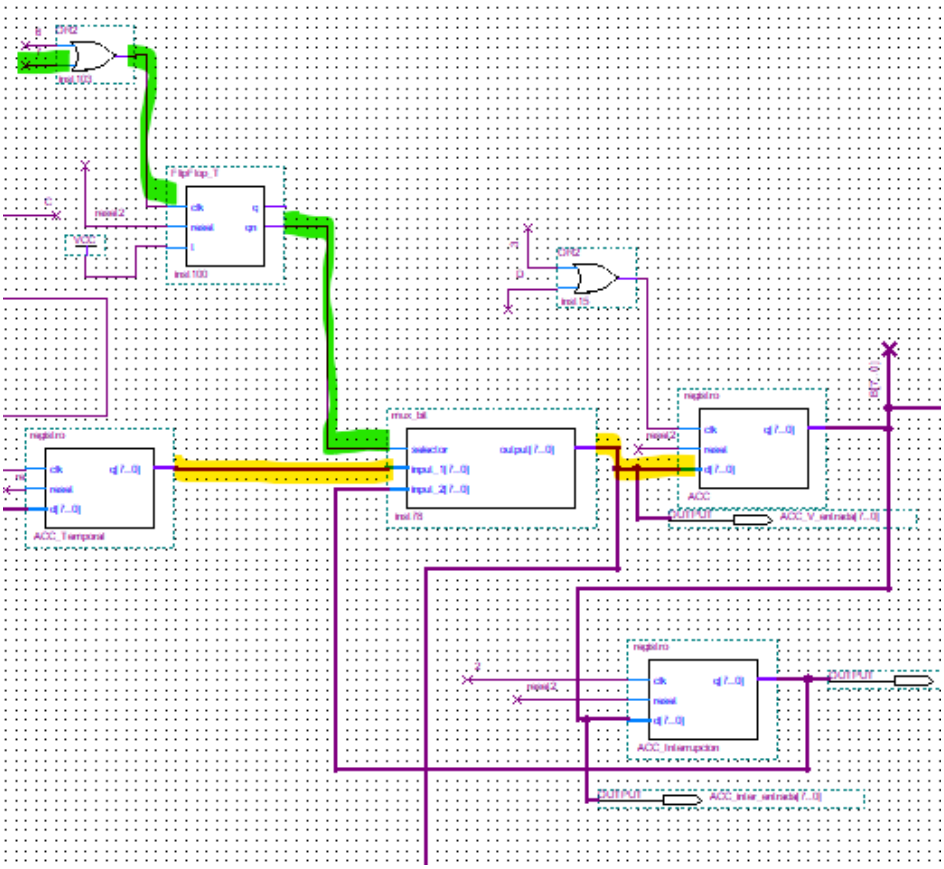

Figura 31: Diseño de bloque M en quartus

Ahora que la entrada de ACC está conectado con la salida de ACCtemp se pasa al siguiente estado.

## 4.13. Seleccionar la dirección de la rutina de interrupcción,mover el valor de 1 a 0 del pin load del PC y reanudar el funcionamiento del GCM

Este proceso se ejecutó antes en la rutina 4, por lo que se repite. Los registro T, permite que al volver ejecutar el estado, los valores guardados de los registros pasen de 1 a 0.

## 4.14. Monitorear flanco de subida del GPIO

Se termina el proceso y se vuelve a un estado 0, en donde se monitorea nuevamente algún flanco de subida.

# 5. Maquina de estado del NVIC

Teniendo en cuenta la lista de tareas a cumplir y los estados usados en cada una, se concluye que es necesario contar con 8 estados, y dos entradas;GCMpausa y GPIO.

El comportamiento de la maquina de estados es el siguiente.

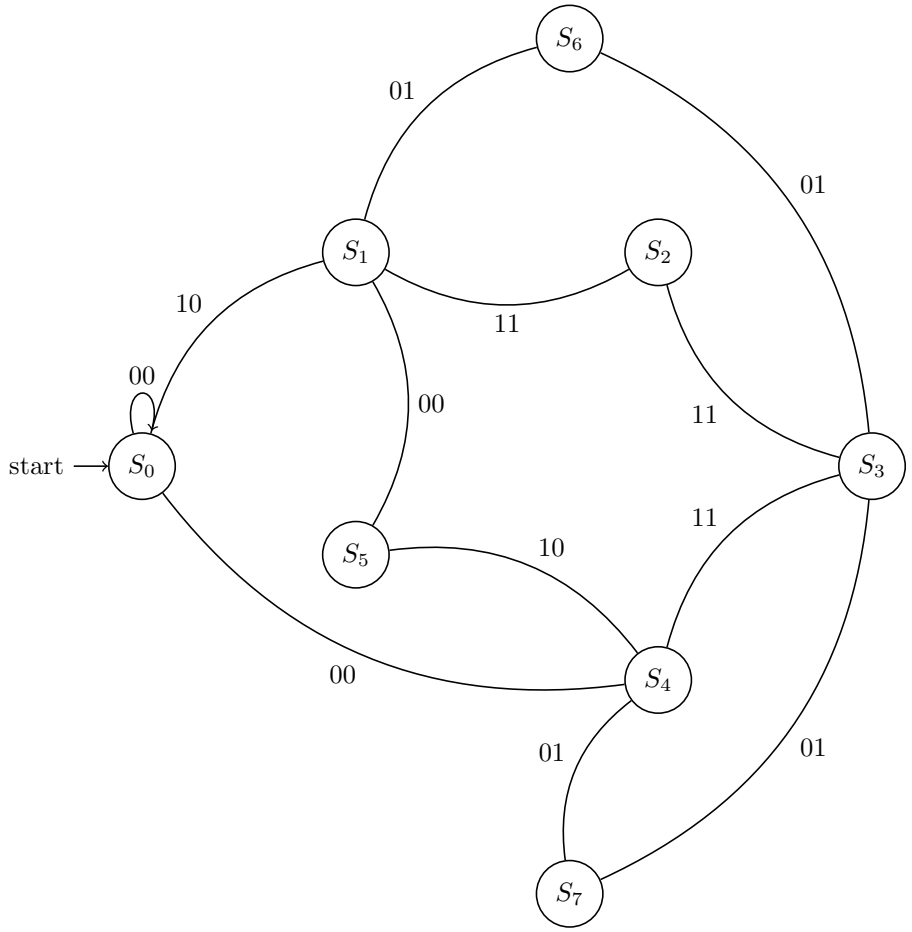


Figura 32: Diagrama de estados del NVIC.

Cómo resumen del Tema 4, se muestra una tabla en donde se visualiza los estados, las entradas y las modificaciones en todo el proceso.

| Mapa de operación | | | |
|---|---|---|---|
| Estados | GPIO/GCM | Acciones | valor actual |
| e0 | 10 | Paso al sigueinte estado | Pausa=0,Load=0,SelAd=0, Band=0,SelDa=0 |
| e1 | 11 | Doy un CLK a un registro T con entrada a Vcc con salida al GCM, esta señal pasará de 0 a 1, pidiendo detener el GCM, por tanto solo se pasará al estado siguiente cuando el GCM me devuelva un 1. | Pausa=1,Load=0,SelAd=0, Band=0,SelDa=0 |
| e2 | 11 | Guardo la dirrección del PC y el valor del registro ACC dando un CLK al registro DIR y al Registro $ACC_{inter}$ respectivamente. | Pausa=1,Load=0SelAd=0, Band=0,SelDa=0 |
| e3 | 11 | Mando un CLK al ACC que en momento no es releante,Doy un CLK a un registro T que hace que load pase de 0 a 1, cargando al PC la direccion que sale el MUX, que en este caso es la direccion de interrupcicón | Pausa=1,Load=1SelAd=0, Band=0,SelDa=0 |
| e4 | 10 | Mando un CLK a los registros T selector de la direccion y al registro de pausa para que pase de 0 a 1, y de 1 a 0 respectivvamente. Tambien reseteo el registro de load haciendo que load pase de 1 a o.[Como en el siguiente estado se ejecutará la rutina de interrupción se pasará solo cuando se despause el GCM] . | Pausa=0,Load=0SelAd=1, Band=0,SelDa=0 |
| e5 | 00 | Aqui se ejecuta la rutina de interupcción y al terminar se ejecuta la interupccion que limpia la bandera por medio de un CLK que va a un registro T con entrada en Alta. Esto hace que el GPIO pase a 0 porque ya se atendió la interrupcción | Pausa=0,Load=0SelAd=1, Band=0,SelDa=0 |
| e1 | 01 | Doy un CLK a un registro T con entrada a Vcc con salida al GCM, esta señal pasará de 0 a 1, pidiendo detener el GCM, por tanto solo se pasará al estado siguiente cuando el GCM me devuelva un 1.[Ahora se regresará todo a su lugar] | Pausa=1,Load=0,SelAd=1, Band=0,SelDa=0 |
| e6 | 01 | Envio un CLK al registro T del selector de dato $ACC_{temporal}$ o el $ACC_{inter}$ para pasar del 0 a 1 y elegir como entrada del ACC el $ACC_{inter}$ [Es para cargar el valor que había en el momento de la interrupción] | Pausa=1,Load=0,SelAd=1, Band=0,SelDa=1 |
| e3 | 01 | Mando un CLK al ACC para capturar el dato guardado en el $ACC_{inter}$ ,Doy un CLK a un registro T que hace que load pase de 0 a 1, cargando al PC la direccion que sale el MUX, que en este caso es la direccion de interrupcicón | Pausa=1,Load=1,SelAd=1, Band=0,SelDa=1 |
| e7 | 01 | Envio un CLK al registro T del selector de dato $ACC_{temporal}$ o el $ACC_{inter}$ para pasar del 1 a 0 y elegir como entrada del ACC el $ACC_{temporal}$, tambien reseteo un registro relacionado al proceso de activacion del GPIO | Pausa=1,Load=1,SelAd=1, Band=0,SelDa=0 |
| e4 | 00 | Mando un CLK a los registros T selector de la direccion y al registro de pausa para que pase de 1 a 0, y de 1 a 0 respectivvamente. Tambien reseteo el registro de load haciendo que load pase de 1 a 0.[Como en el siguiente estado se ejecutará la rutina de interrupción se pasará solo cuando se despause el GCM] . | Pausa=0,Load=0,SelAd=0, Band=0,SelDa=0 |
| e0 | 00 | Se mantiene hasta que vuelva a pasar una interrupción. | sPausa=0,Load=0,SelAd=0, Band=0,SelDa=0 |

## 5.1. Maquina de estado del NVIC en Quartus

Después de hacer la tabla de verdad, y obtener las funcione slogicas se describieron por medio de compuertas logicas y FlipFlop tipo D

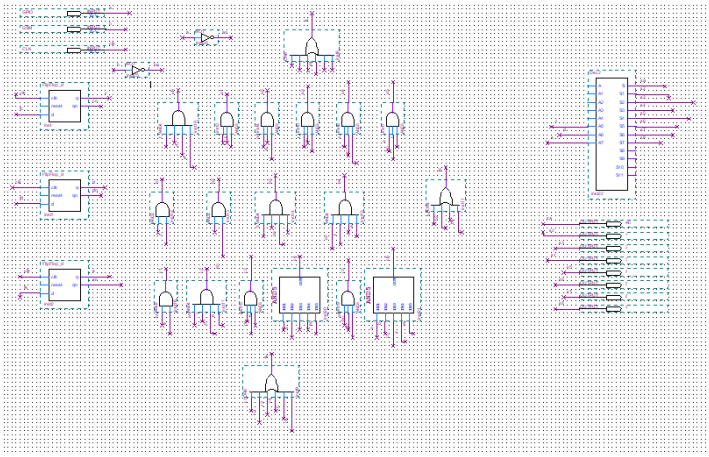

Figura 33: NVIC Quartus

El agregar un decodificador al final del diagrama permite tener señales separadas a la salida.

# 6. Instrucciones

Para agregar un NVIC y GPIO al comportamiento preestablecido del procesador, es indispensable incluir instrucciones para configurar los elementos añadidos.

| Tabla de instrucciones | |
|---|---|
| Instrucción | Acción |
| 0 | XOR |
| 1 | NOT |
| 2 | SUMA |
| 3 | AND |
| 4 | OR |
| 5 | No operación |
| 6 | Cargar |
| 7 | Reset |
| 8 | Deshabilitar Interrupción |
| 9 | Habilitar Interrupción |
| 10 | Limpiar bandera |
| 15 | Habilitar CLK del GPIO |
| 16 | Deshabilitar CLK del GPIO |
| 17 | Configurar GPIO modo OUTPUT |
| 18 | Configurar GPIO modo INPUT |

# 7. Resultados

Se configurará el GPIO y NVIC en las variantes necesarias para demostrar su corecto funcionamiento.

## 7.1. GPIO modo entrada

Configurando el GPIO como entrada para sumar el valor al ACC, se obtiene el siguiente resultado.

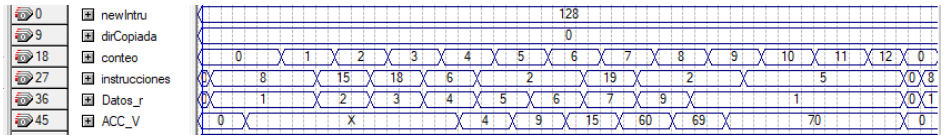


Figura 34: GPIO INPUT

Como se observa el valor del GPIO (60) se carga y después se sumar el valor 9 de la memoria de datos, obteniendo un 69.

## 7.2. GPIO modo salida

Ahora el GPIO se configura como salida para que muestre los valores del registro ACC.

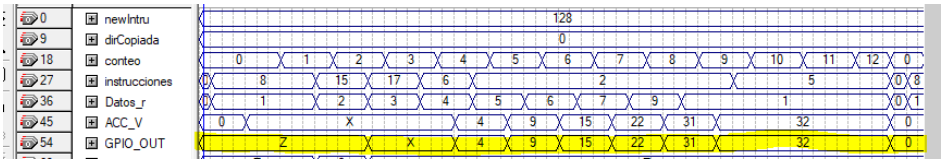


Figura 35: GPIO OUTPUT

Se observa como las señales de salida ACC y GPIO coinciden a partir de la configuración adecuada.

## 7.3. GPIO como interrupción

Ahora el GPIO se configura como entrada y se habilitará la interupcción.

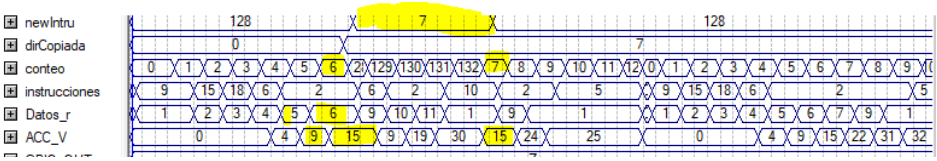


Figura 36: GPIO NVIC

Se observá que se guarda la instrucción más uno, asi como el ACC, para que despues de la rutina de interupccion continue el proceso.

# 8. Conclusiones

En este proyecto, fue de suma importancia seguir un orden estructurado. El uso de diagramas de bloques y de estado permitirá al lector una mejor comprensión. Este proyecto no solo contribuyó a mejorar mi entendimiento de la estructura a bajo nivel de NVIC y GPIO, sino que también me brindó herramientas para el desarrollo y la documentación de futuros proyectos.